Name: B . Sairaj

Roll No: 2503a52162

Batch: 06

Course: AI Assisted Coding

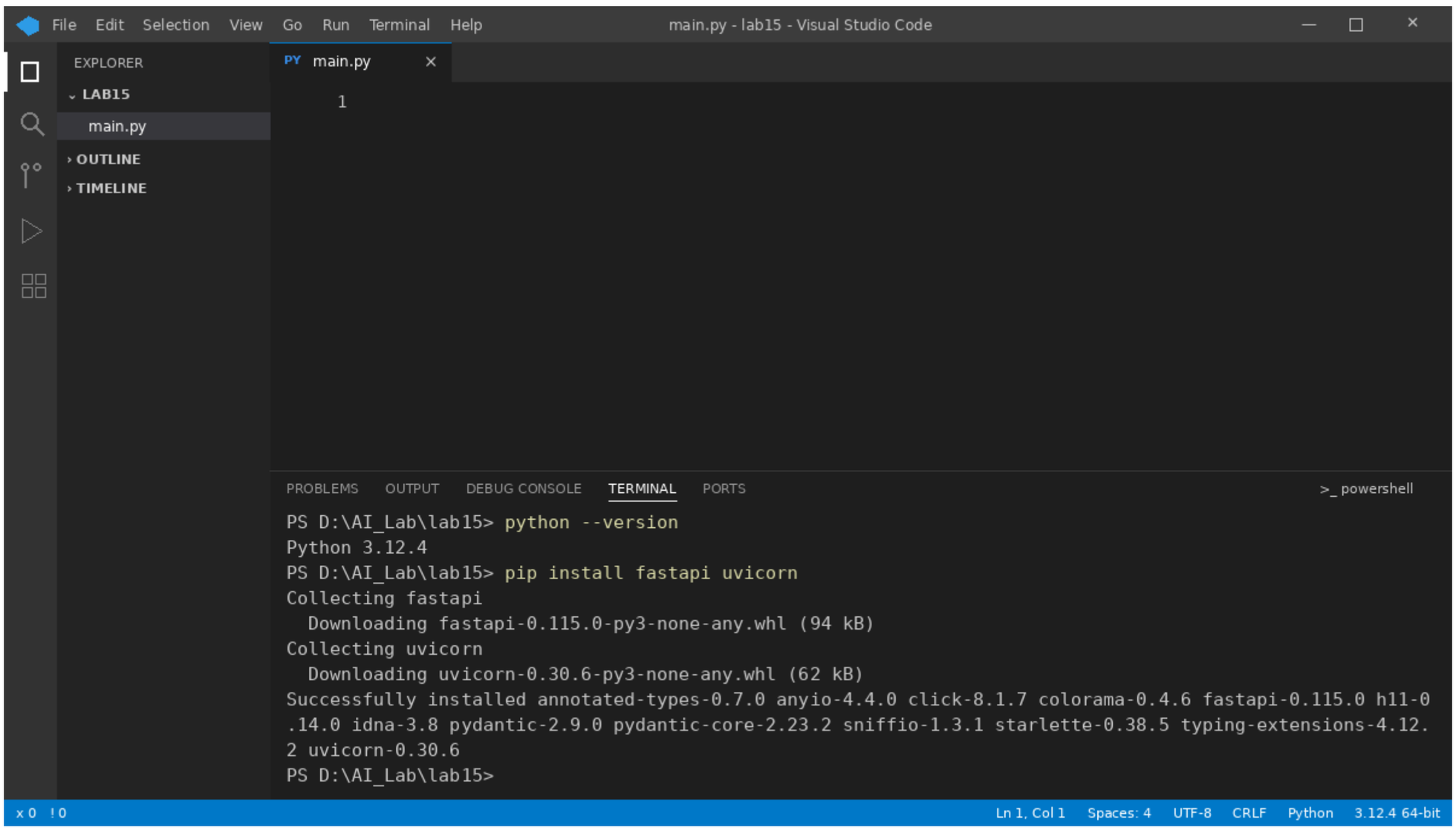
Assignment No: 15 / 24

Lab 15 - Backend API Development: Creating RESTful services with AI

Task 1: Setup FastAPI Backend

Prompt given to the AI:

Generate a basic FastAPI application with one GET endpoint at the root URL "/" which returns the JSON message "Welcome to AI-powered FastAPI". Also give the commands to install FastAPI and Uvicorn and to run the server.



Code (main.py):

```
from fastapi import FastAPI

app = FastAPI()



@app.get("/")
def read_root():
    return {"message": "Welcome to AI-powered FastAPI"}
```

Explanation:

The first line imports the FastAPI class and app = FastAPI() creates the application object. The decorator @app.get("/") connects the function read_root() to GET requests on the root URL. The function simply returns a Python dictionary and FastAPI automatically converts it into a JSON response, so no extra code is needed for that.

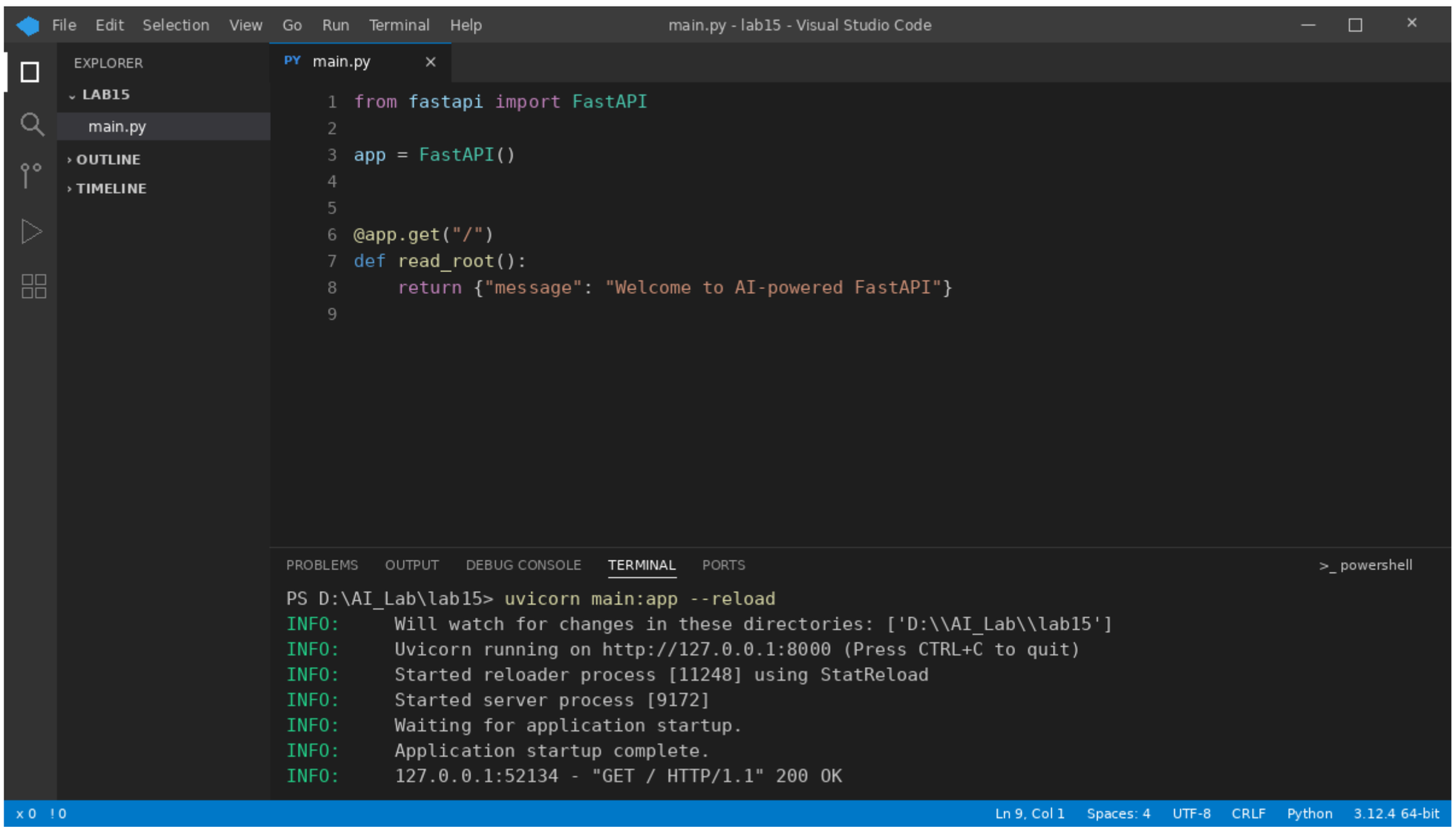
The server is started with the command uvicorn main:app --reload. Here "main" is the file name (main.py), "app" is the FastAPI object inside it, and --reload makes the server restart automatically whenever I save the file.

```
uvicorn main:app --reload
```

Output:

After running the server and opening http://127.0.0.1:8000/ in the browser, the following response is displayed:

{"message": "Welcome to AI-powered FastAPI"}





Task 2: API - Create and Read Items

Prompt given to the AI:

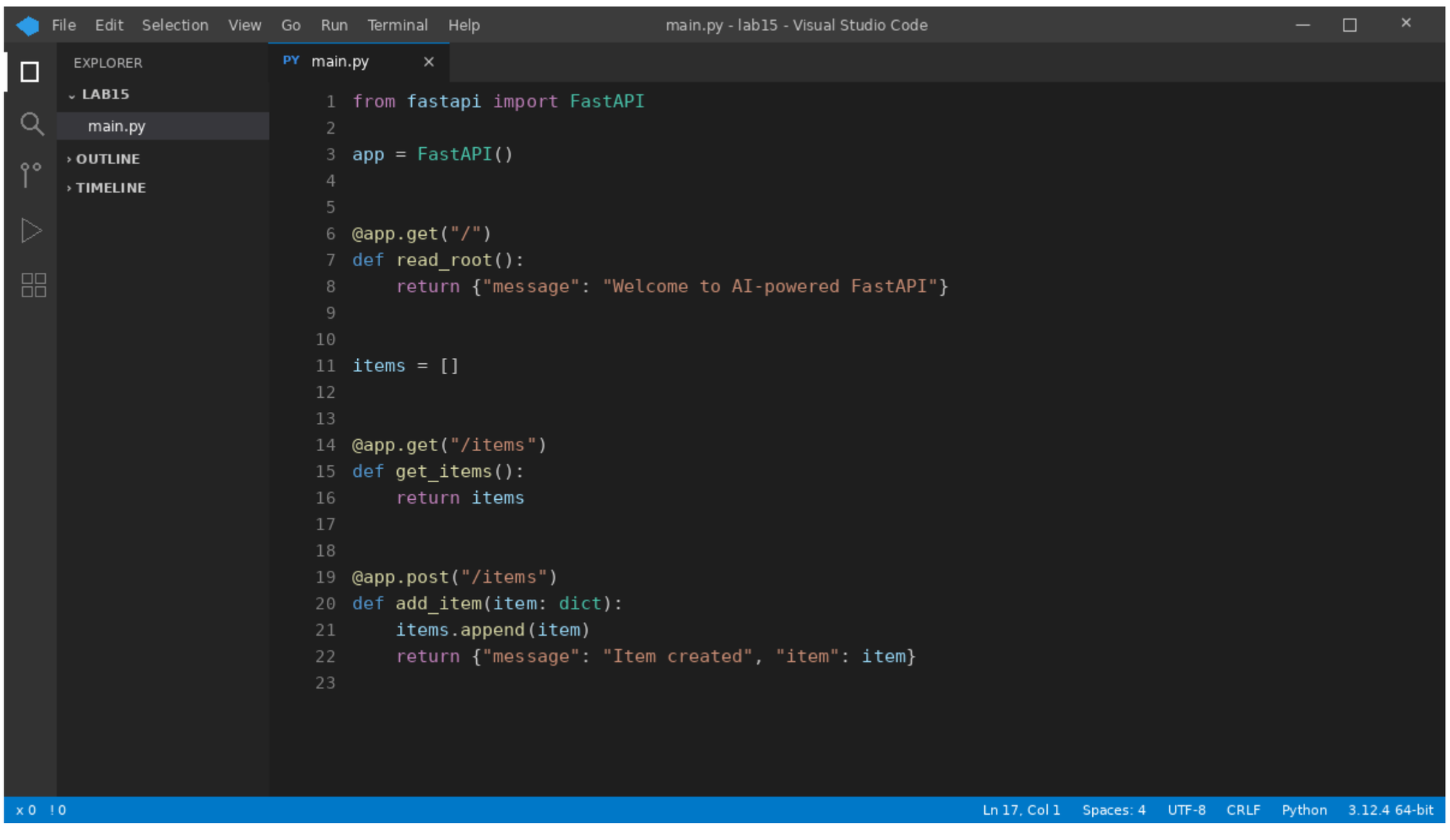
Extend the above FastAPI code with two endpoints for products: GET /items to return all the items and POST /items to add a new item. Use a Python list as temporary storage and show a sample request and response.

Code added to main.py:

```
items = []


@app.get("/items")
def get_items():
    return items


@app.post("/items")
def add_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```



Explanation:

items = [] is an empty list which works as the temporary database. Because it is stored only in memory, all the data is lost when the server restarts. The GET /items endpoint just returns the whole list. In the POST /items endpoint the parameter item: dict tells FastAPI to read the JSON body of the request and give it to the function as a dictionary.

The function appends it to the list and returns a confirmation message along with the item that was added.

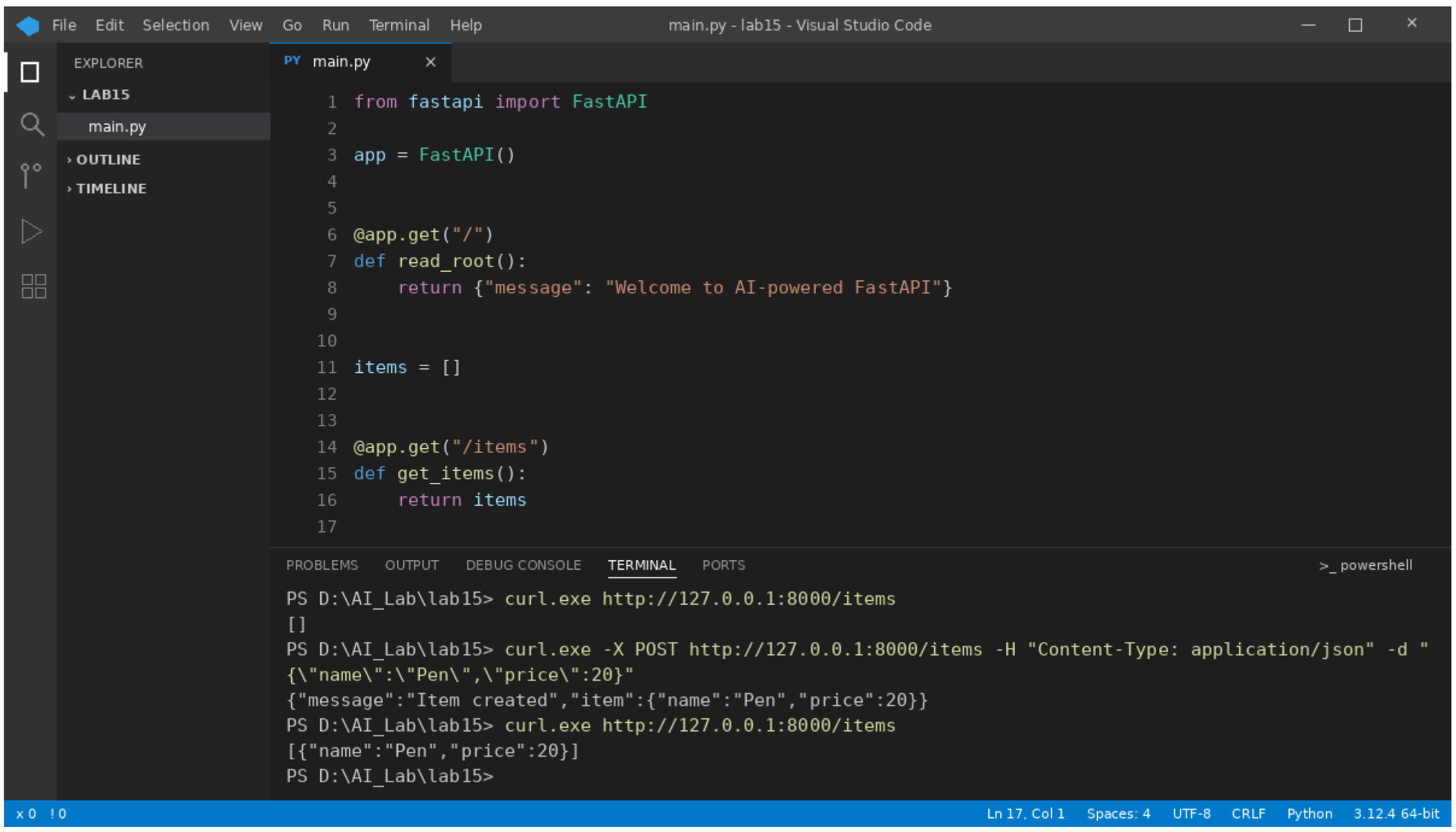
To test the endpoints I used curl.exe from the PowerShell terminal (the double quotes inside the JSON have to be escaped with a backslash in PowerShell).

Output:

GET /items (list is empty at first) ->[]

POST /items with {"name":"Pen","price":20} ->{"message":"Item created","item":{"name":"Pen","price":20}}

GET /items again ->[{"name":"Pen","price":20}]



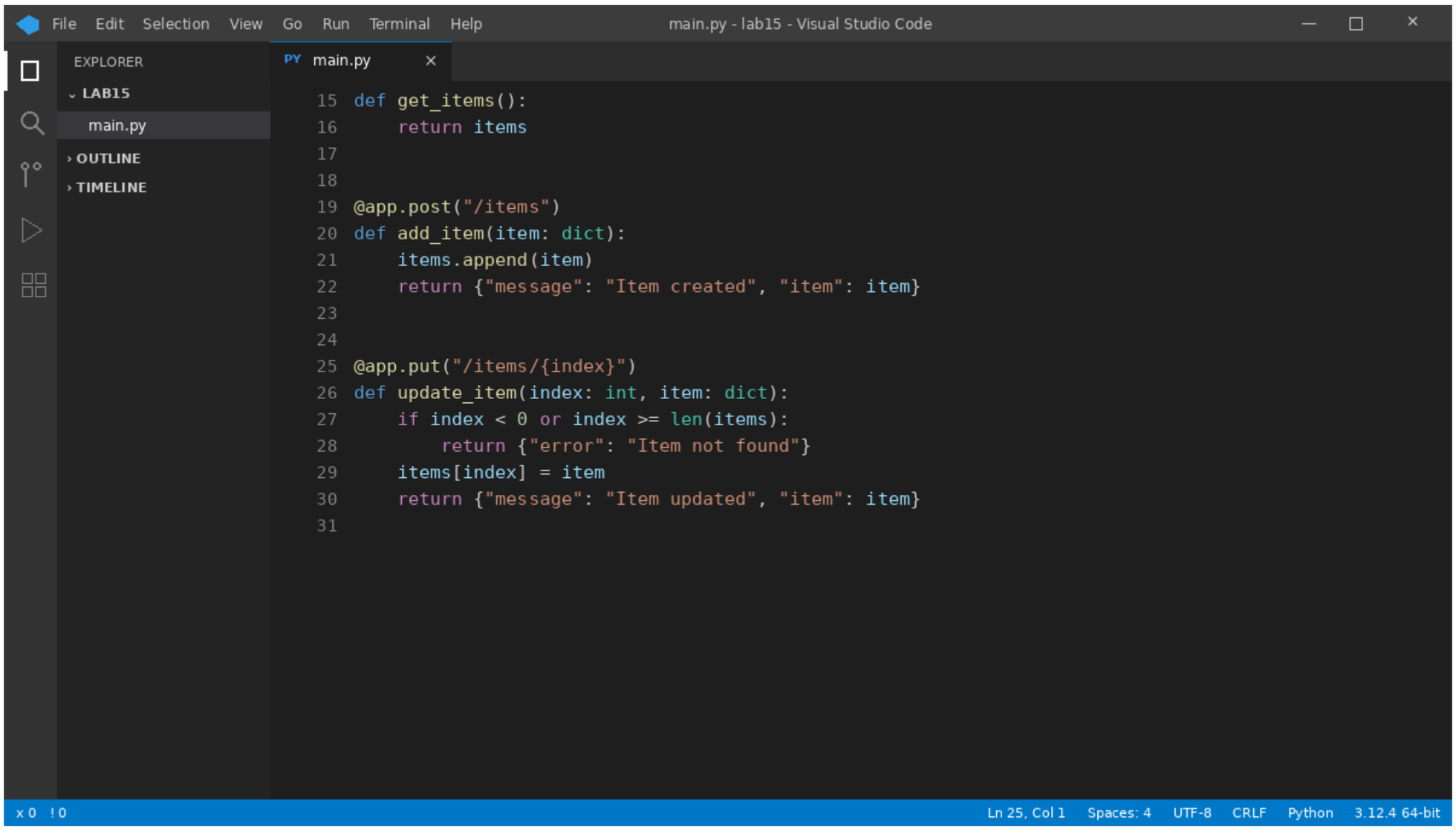
Task 3: Update Existing Item

Prompt given to the AI:

Add a PUT endpoint /items/{index} which updates an existing item using its index in the list. If the index does not exist it should return an error message instead of crashing.

Code added to main.py:

```
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    if index < 0 or index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```



Explanation:

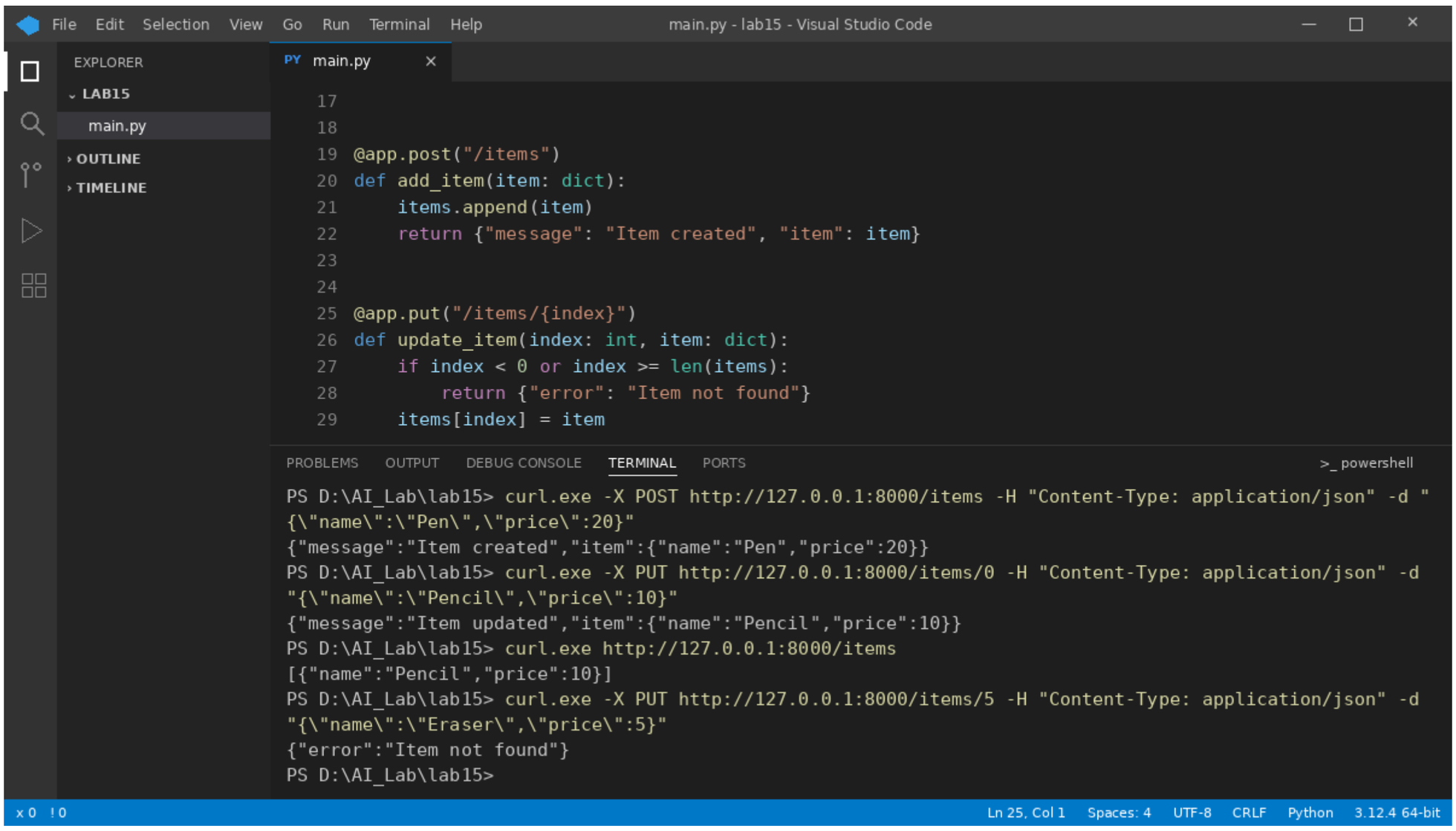
The {index} in the URL is a path parameter and because it is declared as int, FastAPI converts it to an integer and also rejects values that are not numbers. Before updating, the code checks whether the index is valid. The sample code only checked index >= len(items), but I also added index<0 because Python lists accept negative indexes (items[-1] is the last element) and that would update an item the user did not intend. If the index is valid, the old item is replaced with the new data from the request body.

Note: since I started the server with --reload, it restarts every time I save the file and the list becomes empty again. So I added the item once more with POST before testing the update.

Output:

PUT /items/0 with {"name":"Pencil","price":10} ->{"message":"Item updated","item":{"name":"Pencil","price":10}}

PUT /items/5 (index which does not exist) ->{"error":"Item not found"}



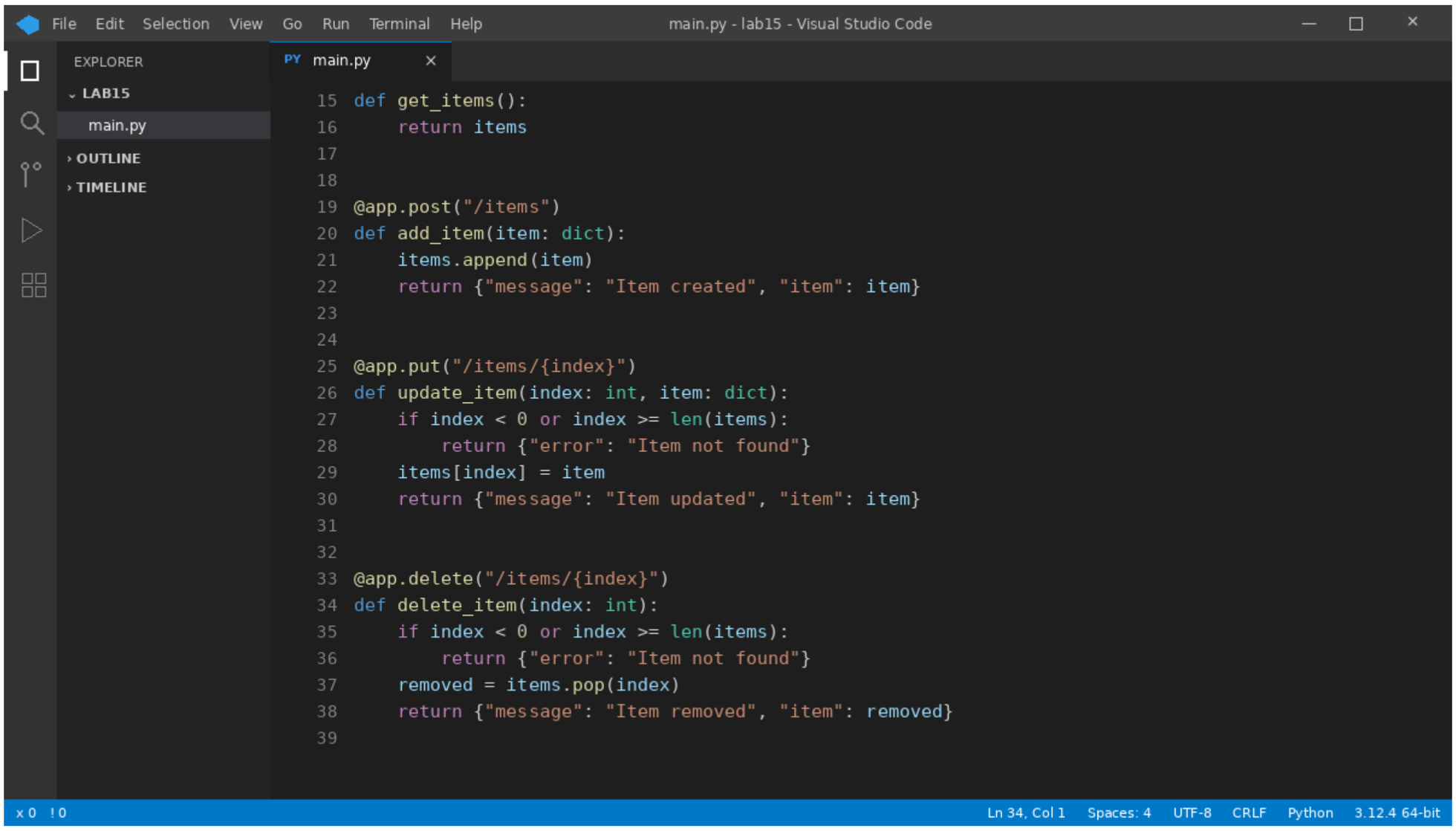
Task 4: Delete Item

Prompt given to the AI:

Add a DELETE endpoint /items/{index} which removes the item at the given index and returns the removed item. Handle the case where the index is invalid.

Code added to main.py:

```
@app.delete("/items/{index}")
def delete_item(index: int):
    if index < 0 or index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```



Explanation:

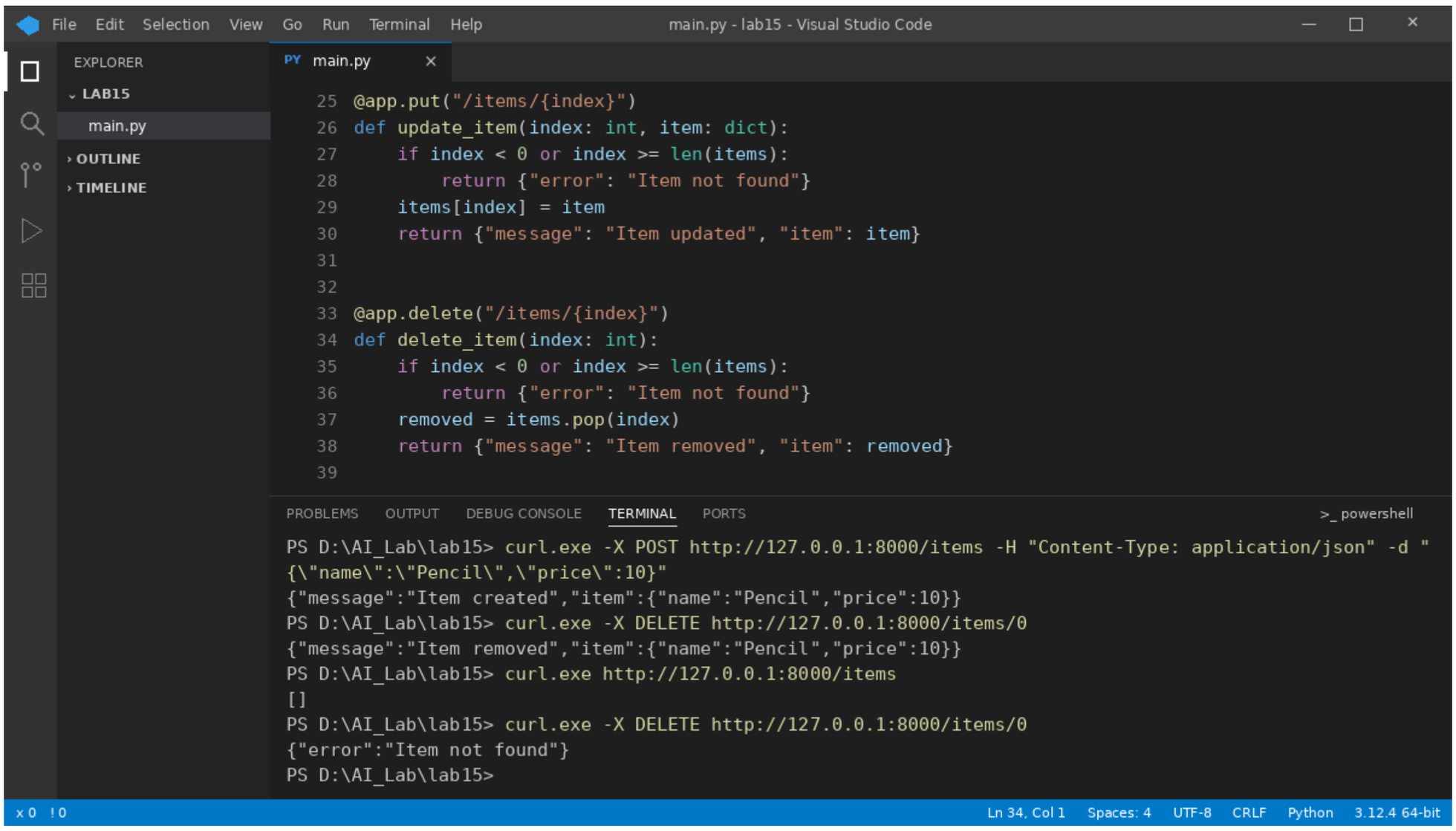
This endpoint uses the same index check as the update endpoint. The list method pop(index) removes the element at that position and also returns it, so I store it in the variable removed and send it back in the response. This way the client can see which item was deleted.

Output:

DELETE /items/0 ->{"message":"Item removed","item":{"name":"Pencil","price":10}}

GET /items after deleting ->[]

DELETE /items/0 again (list is empty now) ->{"error":"Item not found"}



Task 5: Add Auto-Generated Documentation

Prompt given to the AI:

Add inline comments and docstrings to all the endpoints of my FastAPI code. Also set a title, description and version for the app and a short summary for every endpoint so that the Swagger documentation at /docs looks clear.

The assignment sample for this task is written in Flask style (@app.route and jsonify). Since my project uses FastAPI, I did not need Flask-RESTX at all, because FastAPI generates the Swagger UI automatically at /docs (and ReDoc at /redoc). I only had to add docstrings and summaries so the documentation shows proper descriptions.

Final code (main.py):

```
# main.py
# Lab 15 - Backend API Development: Creating RESTful services with AI
# A simple CRUD API built with FastAPI. Data is kept in a Python list (temporary storage).


from fastapi import FastAPI

# Creating the FastAPI application object.
# title, description and version are shown at the top of the Swagger page (/docs).
app = FastAPI(
    title="AI-powered FastAPI",
    description="A simple REST API to create, read, update and delete items.",
    version="1.0.0",
)


# Temporary storage for the items (the data is lost when the server restarts)
items = []
```

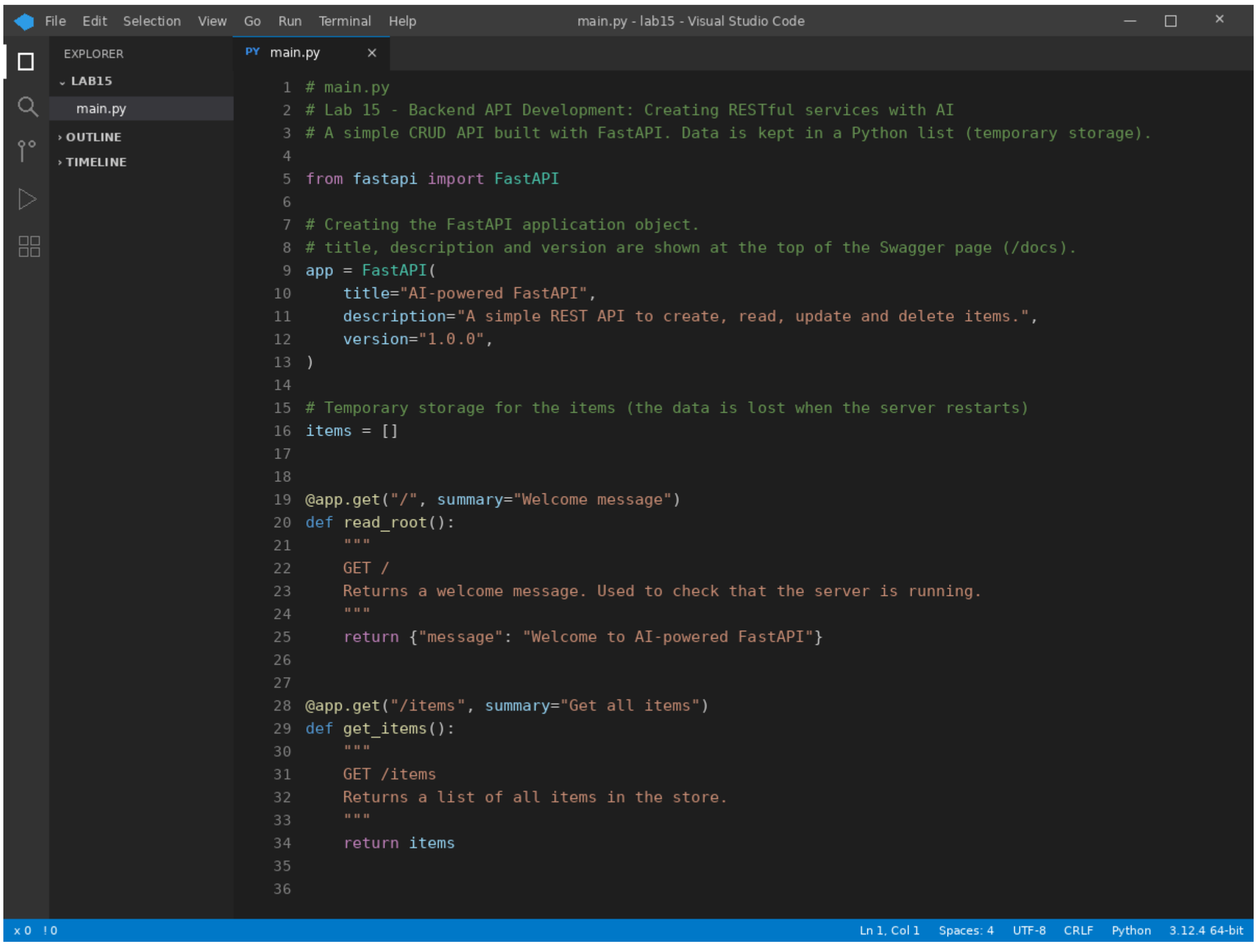
```
@app.get("/", summary="Welcome message")
def read_root():
    """
    GET /
    Returns a welcome message. Used to check that the server is running.
    """
    return {"message": "Welcome to AI-powered FastAPI"}
```

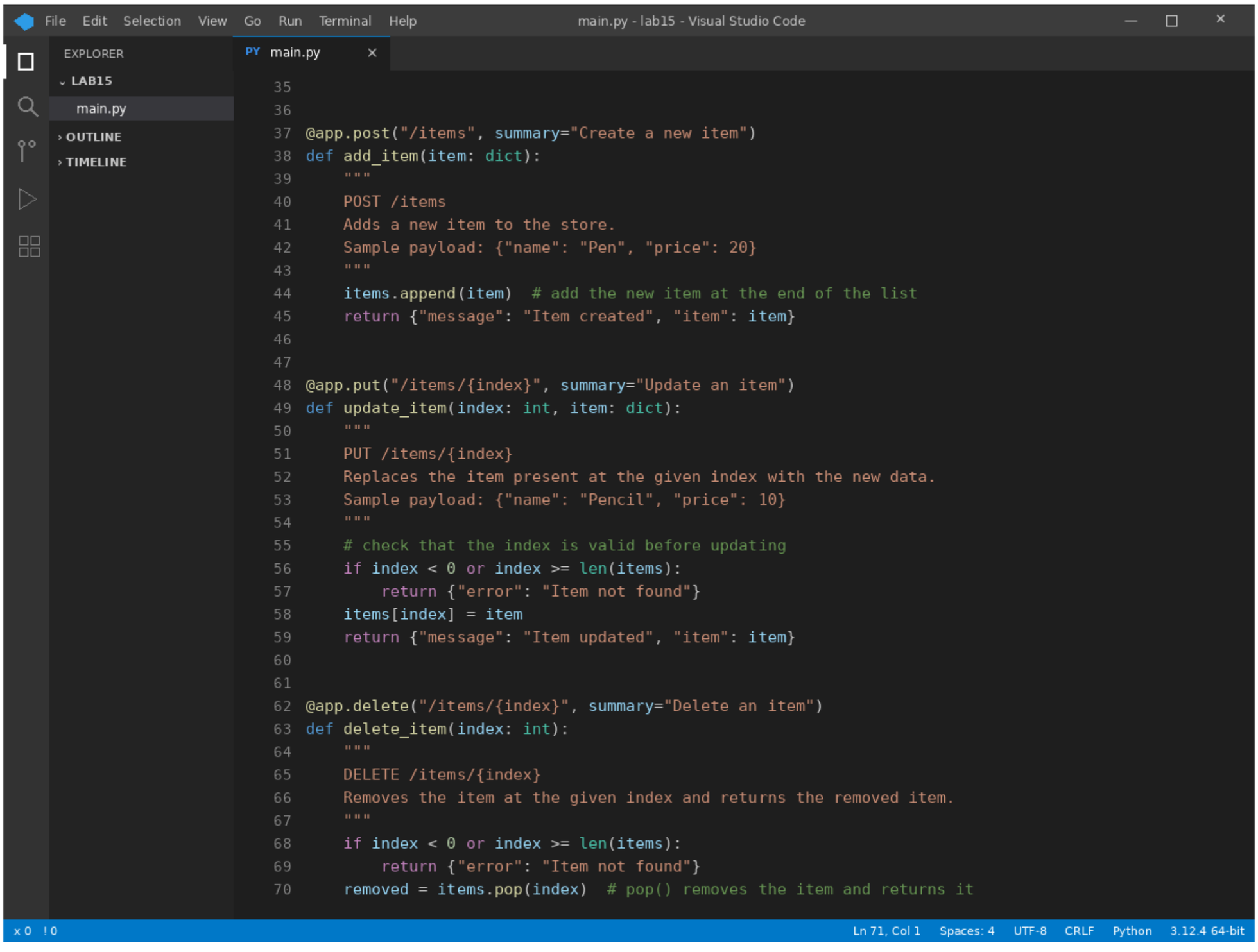
```
@app.get("/items", summary="Get all items")
def get_items():
    """
    GET /items
    Returns a list of all items in the store.
    """
    return items
```

```
@app.post("/items", summary="Create a new item")
def add_item(item: dict):
    """
    POST /items
    Adds a new item to the store.
    Sample payload: {"name": "Pen", "price": 20}
    """
    items.append(item)  # add the new item at the end of the list
    return {"message": "Item created", "item": item}
```

```
@app.put("/items/{index}", summary="Update an item")
def update_item(index: int, item: dict):
    """
    PUT /items/{index}
    Replaces the item present at the given index with the new data.
    Sample payload: {"name": "Pencil", "price": 10}
    """
    # check that the index is valid before updating
    if index < 0 or index >= len(items):
        return {"error": "Item not found"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

```
@app.delete("/items/{index}", summary="Delete an item")
def delete_item(index: int):
    """
    DELETE /items/{index}
    Removes the item at the given index and returns the removed item.
    """
    if index < 0 or index >= len(items):
        return {"error": "Item not found"}
    removed = items.pop(index)  # pop() removes the item and returns it
    return {"message": "Item removed", "item": removed}
```


File Edit Selection View Go Run Terminal Help
main.py - lab15 - Visual Studio Code
EXPLORER
LAB15
main.py
OUTLINE
TIMELINE
main.py
main.py
Lab 15 - Backend API Development: Creating RESTful services with AI
A simple CRUD API built with FastAPI. Data is kept in a Python list (temporary storage).
from fastapi import FastAPI
Creating the FastAPI application object.
title, description and version are shown at the top of the Swagger page (/docs).
app = FastAPI(
title="AI-powered FastAPI",
description="A simple REST API to create, read, update and delete items.",
version="1.0.0",
)
Temporary storage for the items (the data is lost when the server restarts)
items = []
@app.get("/", summary="Welcome message")
def read_root():
"""
GET /
Returns a welcome message. Used to check that the server is running.
"""
return {"message": "Welcome to AI-powered FastAPI"}
@app.get("/items", summary="Get all items")
def get_items():
"""
GET /items
Returns a list of all items in the store.
"""
return items
x 0 ! 0
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.4 64-bit



Explanation:

The title, description and version given while creating FastAPI() are shown at the top of the Swagger page. The summary argument of each decorator is the short heading displayed next to the endpoint path, and the docstring inside the function is shown as the full description when the endpoint is expanded. I also wrote the sample payload inside the docstrings of POST and PUT so that anyone reading the documentation knows what JSON has to be sent. The # comments explain the important lines of the code.

Output:

After restarting the server, I opened http://127.0.0.1:8000/docs in the browser. All five endpoints (GET /, GET /items, POST /items, PUT /items/{index}, DELETE /items/{index}) are listed with their methods and descriptions. I expanded the POST /items endpoint, used the "Try it out" option with the sample payload {"name": "Pen", "price": 20} and clicked Execute. The server returned status code 200 with the response body {"message": "Item created", "item": {"name": "Pen", "price": 20}}.

AI-powered FastAPI - Swagger UI

127.0.0.1:8000/docs

AI-powered FastAPI 1.0.0 OAS 3.1

/openapi.json

A simple REST API to create, read, update and delete items.

default

GET	/	Welcome message
GET	/items	Get all items
POST	/items	Create a new item
PUT	/items/{index}	Update an item
DELETE	/items/{index}	Delete an item

AI-powered FastAPI - Swagger UI

127.0.0.1:8000/docs#/default/add_item_items_post

POST /items Create a new item

POST /items

Adds a new item to the store.
Sample payload: {"name": "Pen", "price": 20}

Parameters Cancel

No parameters

Request body required application/json

```
{
  "name": "Pen",
  "price": 20
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/items' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "name": "Pen",
  "price": 20
}'
```

Request URL

```
http://127.0.0.1:8000/items
```

Server response

Code	Details
200	Response body

```
{
  "message": "Item created",
  "item": {
    "name": "Pen",
    "price": 20
  }
}
```

Conclusion

In this lab I created a REST API with FastAPI and implemented all four CRUD operations on a list of items using the GET, POST, PUT and DELETE methods. I learned how path parameters and JSON request bodies work, how to handle an invalid index, and how FastAPI generates interactive documentation automatically from the code. Using an AI tool made it faster to generate the base code, but I still had to test every endpoint and fix small things myself, such as the negative index check.